LITTLE NEST-ECOMMERCE WEBSITE

A SUMMER INTERNSHIP REPORT

Submitted by

HARINI. S – 810023205099



DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY,

TIRUCHIRAPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY, TIRUCHIRAPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

This is to certify this Summer Internship project report **"LITTLE NEST-ECOMMERCE WEBSITE"** is the Bonafide work of **HARINI S (810023205099)** has satisfactory completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026.**

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____.

ABSTRACT

The Little Nest E-Commerce Website is a Java Full Stack web application developed to provide a modern, secure, and user-friendly online shopping platform. The system enables customers to browse products, view detailed product information, search for products, add items to a shopping cart, register, log in, and place orders through an interactive web interface. The project was developed as part of an internship to gain practical experience in full-stack web application development using modern technologies.

The frontend of the application is built using React.js, which provides a responsive and dynamic user interface. The backend is developed using Spring Boot, exposing RESTful APIs that handle business logic, product management, order processing, customer authentication, and contact requests. Spring Data JPA is used for database operations. The application uses MySQL as the relational database for storing product, user, order, and contact information

The system includes several essential e-commerce features such as product listing, product detail pages, search functionality, shopping cart management, user registration and login, checkout, order creation, and a contact form for customer inquiries. The shopping cart and user session are maintained using browser local storage, while the backend validates registered users before processing orders.

Overall, This project demonstrates the practical implementation of Java Full Stack development by integrating frontend technologies with a secure Spring Boot backend and a MySQL database. It provides hands-on experience in REST API development, database management, authentication, responsive web design.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVE OF THE INTERNSHIP	1-
2	COMPANY PROFILE	
3	DAILY WORK REPORT	
4	BLOCK DIAGRAM	
5	TRAVEL AGENCY MANAGEMENT SYSTEM 5.1 EXISTING SYSTEM 5.2 PROPOSED SYSTEM 5.3 CAPABILIITIES	
6	PROPOSED METHOD 6.1 USER AUTHENTICATION 6.2 PRODUCT MANAGEMENT 6.3 SHOPPING CART AND ORDER MANAGEMENT 6.4 CONTACT MANAGEMENT 6.5 SYSTEM MANAGEMENT	
7	ALGORITHM 7.1 USER LOGIN 7.2 USER REGISTRATION 7.3 PRODUCT SEARCH AND SHOPPING CART 7.4 ORDER PROCESSING 7.5 CONTACT MANAGEMENT	
8	CODE IMPLEMENTATION	
9	SAMPLE OUTPUT	
10	ANALYSIS	
11	CONCLUSION	
12	FUTURE WORK	
13	REFERENCES	

1. INTRODUCTION

The rapid advancement of internet technologies and digital communication has significantly transformed the way people purchase goods and services. E-commerce has become an important part of modern business by providing customers with the convenience of shopping from anywhere at any time. It offers several advantages such as wider product availability, secure transactions, reduced shopping time, and improved customer satisfaction. As online shopping continues to grow, businesses require efficient and reliable web applications to meet customer expectations.

An e-commerce website serves as a digital marketplace where customers can explore products, compare prices, place orders, and communicate with sellers without visiting physical stores. Such systems help businesses expand their reach, improve customer engagement, and automate various business operations, including product management, order processing, and customer support. This has made e-commerce one of the fastest-growing sectors in today's digital economy.

The **Little Nest E-Commerce Website** is a Java Full Stack web application developed to provide a complete and user-friendly online shopping platform. The application enables users to register, log in, browse products, search for specific items, view detailed product information, add products to a shopping cart, and place orders through a simple and interactive interface. It also includes a contact feature that allows customers to communicate directly with the store.

The frontend of the application is developed using **React.js**, which provides a responsive and interactive user interface that enhances the overall user experience. The backend is built using **Spring Boot**, which manages business logic and exposes RESTful APIs for communication between the frontend and the database. This client-server architecture ensures efficient data exchange, better maintainability, and high system performance.

The project uses **Spring Data JPA** to simplify database operations and **MySQL** as the relational database management system for storing product details, customer information, orders, and contact messages. To ensure secure authentication, **Spring Security** with **BCrypt Password Encoder** is implemented for password encryption. This approach enhances data security by protecting user credentials and preventing unauthorized access.

The application provides several important features, including product listing, product search, shopping cart management, user registration, secure login, checkout, order creation, and a contact form. The shopping cart is maintained using browser local storage, while the backend verifies registered users before processing orders. These features work together to provide customers with a smooth, secure, and reliable online shopping experience.

The system follows a modular and scalable architecture, making it easy to maintain and enhance. Each module performs a specific function, allowing developers to update or improve individual components without affecting the entire application. The project also provides a strong foundation for implementing future enhancements such as JWT-based authentication, online payment gateways, order tracking, inventory management, product reviews, and cloud deployment.

Overall, this internship project is to apply the concepts of Java Full Stack Development in building a real-world web application. During the development, practical knowledge was gained in frontend development, backend programming, REST API integration, database management, authentication, debugging, and project testing. This project not only strengthened technical skills but also provided valuable experience in designing, developing, and integrating modern web technologies to create a scalable, secure, and efficient e-commerce system.

1.1 OBJECTIVES OF THE INTERNSHIP

The main objective of the **Little Nest E-Commerce Website** is to develop a secure and user-friendly online shopping platform using Java Full Stack technologies. The application enables users to register, log in, browse products, search for items, add products to the cart, and place orders efficiently. It also aims to provide secure authentication, efficient database management, and smooth communication between the frontend and backend through REST APIs.

The specific objectives are:

- To develop a responsive e-commerce website using React.js and Spring Boot.
- To implement secure user registration and login using Spring Security with BCrypt password encryption.
- To provide product browsing, search, and detailed product information.
- To enable shopping cart management and order placement for registered users.
- To store and manage product, user, order, and contact information using MySQL.
- To develop a modular and scalable application that can be easily enhanced with future features such as online payment, order tracking, and JWT-based authentication.

1.2 MOTIVATION

The motivation behind developing the **Little Nest E-Commerce Website** is the increasing demand for online shopping platforms that offer convenience, accessibility, and secure transactions. Traditional shopping methods often require customers to visit physical stores, whereas e-commerce applications allow users to explore products and complete purchases from anywhere at any time.

This project was undertaken as part of the internship to gain practical experience in Java Full Stack Development and to understand how modern web applications are designed and implemented. It provided an opportunity to apply theoretical concepts such as frontend development, backend programming, REST API integration, database management, and user authentication in a real-world application.

The project also helped in improving problem-solving skills, software development practices, and knowledge of industry-standard technologies used in building scalable web applications.

2. COMPANY PROFILE



About the Company:

Astonish InfoTech Private Limited is a leading **Information Technology (IT)** company located in **Tiruchirappalli (Trichy), Tamil Nadu, India**. The company specializes in providing high-quality software solutions, web application development, mobile application development, and digital technology services to clients from various business sectors. Established on **25 February 2010**, the company has built its reputation by delivering reliable, innovative, and customer-oriented software products that meet modern business requirements.

The primary objective of Astonish InfoTech is to develop software solutions that improve business productivity, simplify business operations, and provide user-friendly digital services. The company follows modern software development methodologies and focuses on delivering scalable, secure, and efficient applications using the latest technologies. It also emphasizes quality assurance, customer satisfaction, and continuous technological improvement throughout the software development process.

Apart from software development, the company actively provides **industrial training and internship programs** for engineering students and fresh graduates. These programs are designed to bridge the gap between academic learning and industry requirements by providing practical exposure to real-time projects. During the internship, students gain hands-on experience in software development, database management, frontend and backend integration, REST API development, software testing, and project deployment under the guidance of experienced professionals.

As part of my internship at Astonish InfoTech Private Limited, I developed a **Ecommerce website** using **Java Full Stack** technologies. The project involved designing responsive user interfaces, developing backend services using **Spring Boot**, integrating the **MySQL** database, implementing REST APIs, and testing the application. This internship helped me improve my technical knowledge, programming skills, problem-solving abilities, and understanding of the complete Software Development Life Cycle (SDLC).

The internship experience also provided practical knowledge of working in a professional software development environment. It enhanced my ability to understand client requirements, develop real-world applications, work with modern development tools, and follow industry standards while building software solutions. Overall, Astonish InfoTech Private Limited provided an

excellent platform to apply theoretical concepts in practical situations and strengthen my skills in **Java Full Stack Development**.

Company Details:

- **Company Name:** Astonish InfoTech Private Limited
- **Established:** 25 February 2010
- **Industry:** Information Technology (IT) Services and Software Development
- **Location:** Technology Park (BUTP), Bharathidasan University Campus, Kajamalai, Tiruchirappalli, Tamil Nadu
- **Development Technologies:** Java, Spring Boot, HTML, CSS, JavaScript, Bootstrap, MySQL, REST APIs, React.
- **Major Services:** Software Development, Web Development, Mobile Application Development, Database Solutions, Digital Marketing, Web Hosting, Technical Training
- **Internship Training:** Java Full Stack Development, Web Application Development, Database Management, Software Testing, Real-Time Projects
- **Development Approach:** Customer-oriented, scalable, secure, and industry-standard software solutions
- **Project Completed During Internship:** Ecommerce Website (Java Full Stack)

Services Offered

- Software Development
- Web Application Development
- Website Design and Development
- Mobile Application Development
- Java Full Stack Development
- Database Design and Management
- REST API Development
- Digital Marketing and SEO
- Domain Registration and Web Hosting
- Technical Internship and Industrial Training
- Software Maintenance and Technical Support

Technologies Used by the Company

- **Programming Languages:** Java, JavaScript
- **Frontend:** React js, html, CSS, JavaScript.
- **Backend:** Java Spring Boot
- **Database:** MySQL
- **Development Tools:** IntelliJ IDEA, VS Code
- **Build Tool:** Apache Maven,Nodejs.
- **Version Control:** Git and GitHub
- **API Technology:** RESTful Web Services

3. DAILY WORK REPORT

Day 1: Attended the internship orientation session and understood the project guidelines, development process, and evaluation criteria. Finalized the project topic **"Little Nest – ECommerce Website"** with the mentor.

Day 2: Conducted requirement analysis by identifying the stakeholders (Customer and Administrator). Gathered the functional and non-functional requirements for the e-commerce application.

Day 3: Performed a literature review on existing e-commerce platforms. Studied their features, advantages, and limitations to understand current online shopping systems.

Day 4: Defined the project objectives, scope, and overall system requirements. Prepared the list of software tools and technologies required for development.

Day 5: Set up the development environment by installing **JDK 17, IntelliJ IDEA, Apache Maven, MySQL, Node.js, Git,** and Visual Studio Code. Verified the installation by creating a sample Spring Boot project.

Day 6: Studied the fundamentals of Spring Boot, including project structure, annotations, REST APIs, dependency injection, and the MVC architecture.

Day 7: Designed the overall system architecture using a client-server model. Planned the interaction between the React frontend, Spring Boot backend, and MySQL database.

Day 8: Designed the database schema by identifying the required entities such as **User, Product, Order, Order Item,** and **Contact Message** along with their attributes.

Day 9: Prepared the Entity-Relationship (ER) Diagram showing the relationships among users, products, orders, order items, and contact messages.

Day 10: Designed wireframes for the Home, Login, Registration, Product Listing, Product Details, Cart, and Contact pages to provide a responsive user interface.

Day 11: Developed the frontend layout using React.js, HTML, CSS, and React Router. Created the Home page with navigation, hero section, featured products, and footer.

Day 12: Developed the Login page with username and password validation. Implemented the user interface and integrated basic client-side validation.

Day 13: Developed the Registration page with user details such as username, email, and password. Added input validation and integrated the registration API.

Day 14: Initialized the Spring Boot backend project using Maven and organized the project into controller, service, repository, configuration, and model packages.

Day 15: Created the entity classes including Product, User, StoreOrder, OrderItem, and ContactMessage using Spring Data JPA.

Day 16: Developed the Product Module by implementing the Product Controller, Repository, and APIs for retrieving product lists and product details.

Day 17: Implemented the Authentication Module by developing the Register and Login REST APIs using Spring Security and BCrypt password encryption.

Day 18: Tested the Product, Registration, and Login APIs using Postman. Verified successful responses and handled validation errors.

Day 19: Developed the Product Details page and integrated it with the backend REST API to display complete product information.

Day 20: Implemented the Shopping Cart module, enabling users to add, remove, and update product quantities. Stored cart information using browser localStorage.

Day 21: Connected the React frontend with the Spring Boot backend using Axios. Integrated product listing, product details, and cart functionalities with REST APIs.

Day 22: Developed the Order Module by implementing order creation, order item management, and server-side validation to ensure only registered users can place orders.

Day 23: Designed and integrated the Checkout page, allowing customers to review cart items, provide order details, and submit purchase requests.

Day 24: Developed the Contact Module by implementing the Contact Controller and Contact Message entity. Enabled customers to send queries through the website.

Day 25: Integrated the Contact page with the backend REST API and verified successful storage of customer messages in the MySQL database.

Day 26: Enhanced application security by implementing password encryption using BCrypt and verified user authentication before order placement.

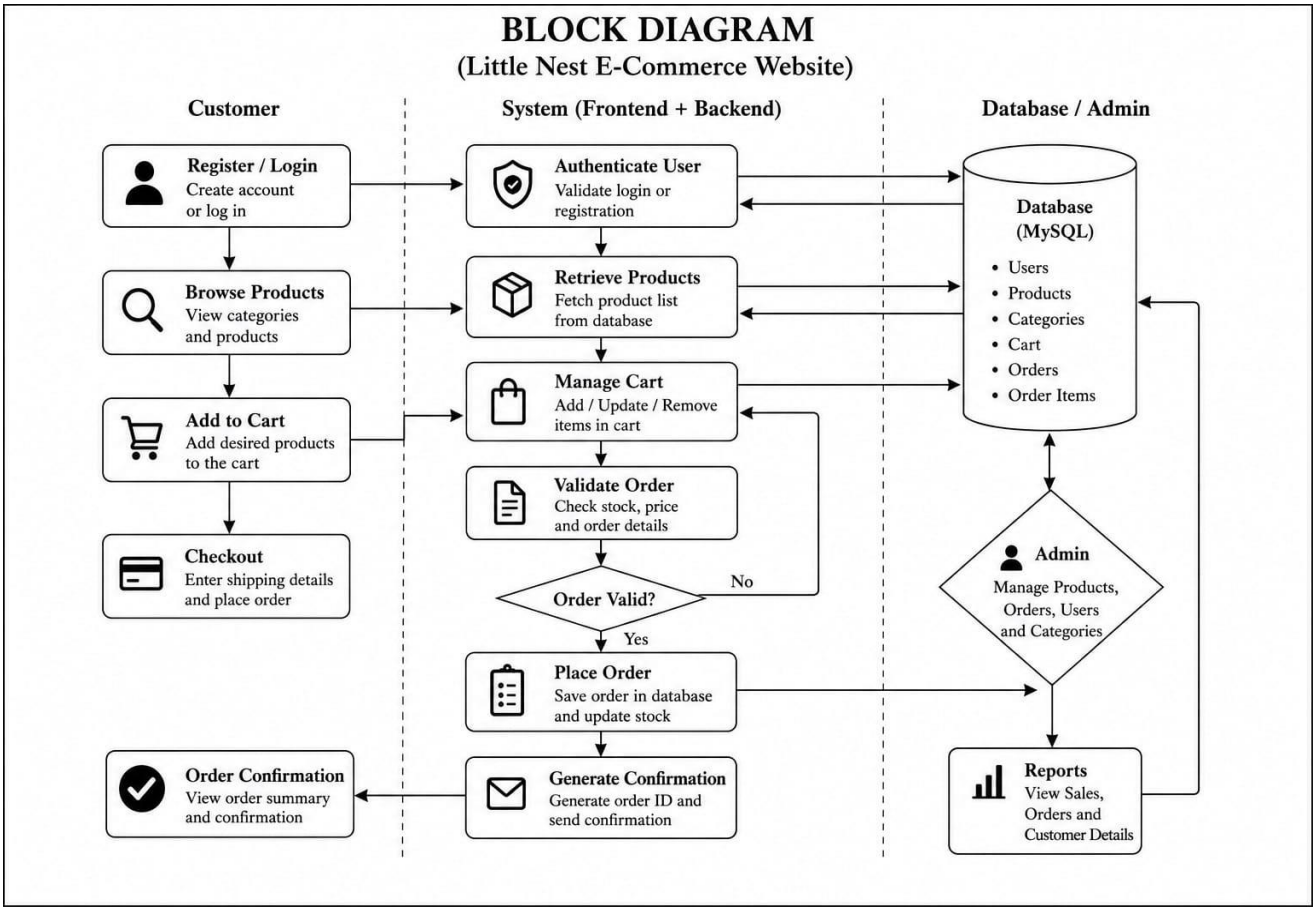
Day 27: Improved the frontend by enhancing responsiveness, refining the user interface, handling exceptions, and displaying user-friendly validation messages.

Day 28: Performed end-to-end integration testing to verify the interaction between the React frontend, Spring Boot backend, and MySQL database. Fixed identified issues.

Day 29: Conducted comprehensive testing of all project modules including Product Management, User Authentication, Shopping Cart, Checkout, Contact, and Order Management. Verified application functionality under different scenarios.

Day 30: Completed final testing, resolved remaining bugs, captured project screenshots, prepared project documentation, and compiled the final internship report for submission.

4. BLOCK DIAGRAM



5. Little Nest -Ecommerce Website

5.1 Existing System

Most traditional retail businesses rely on physical stores or basic online platforms with limited functionality for managing products, customers, and orders. Customers often need to visit stores to purchase products or use websites that lack advanced features such as secure authentication, real-time order management, and an interactive shopping experience. These limitations reduce customer convenience and make business operations less efficient.

The existing approach suffers from several limitations:

- **Manual order management** – Orders are often processed manually, increasing the chances of errors, delays, and inaccurate records.

- **Limited product accessibility** – Customers cannot easily browse,

search, or compare products through a well-organized online platform.

- **Lack of secure authentication** – Some systems do not provide secure user registration and login, making customer data vulnerable.

- **Poor shopping experience** – Many traditional systems lack features such as shopping carts, product search, and detailed product information.

- **Inefficient data management** – Customer details, product information, and order records are maintained separately, making data retrieval difficult.

- **Limited customer communication** – Customers have limited options to contact the business or submit inquiries through the system.

These limitations reduce operational efficiency and customer satisfaction, creating the need for a secure, responsive, and centralized e-commerce application that simplifies online shopping and business management.

5.2 Proposed System

The proposed **Little Nest E-Commerce Website** is a full-stack web application that provides a secure and centralized online shopping platform for customers and administrators. It replaces traditional shopping methods with a modern web-based system that allows users to browse products, register, log in, manage their shopping cart, and place orders conveniently from anywhere.

The system is developed using **React.js** for the frontend, **Spring Boot** for the backend, **Spring Data JPA** for database operations, and **MySQL** as the database. The application is designed to be responsive, scalable, and easy to maintain.

Key features of the proposed system include:

- **Centralized data management** – All product, user, order, and contact information is stored and managed in a single MySQL database.
- **Secure user authentication** – Customers can register and log in securely with encrypted passwords using Spring Security and BCrypt.
- **Interactive shopping experience** – Users can browse products, search for items, view product details, add items to the shopping cart, and place orders easily.
- **Order management system** – Registered users can complete the checkout process, and the backend validates user authentication before creating orders.
- **Integrated contact module** – Customers can send inquiries or feedback through the contact page, and the messages are securely stored in the database.
- **Responsive web interface** – The React-based frontend provides a smooth and user-friendly experience across different devices.
- **Scalable architecture** – The modular design allows future enhancements such as JWT authentication, online payment integration, order tracking, inventory management, product reviews, and cloud deployment.

By automating the shopping and order management process, the proposed system improves operational efficiency, enhances customer satisfaction, reduces manual effort, and provides a secure and reliable online shopping experience.

5.3 Capabilities

The Little Nest E-Commerce Website offers the following key capabilities:

- **User Management** – Provides secure user registration, login, and authentication using Spring Security with BCrypt password encryption.
- **Product Management** – Displays product listings and detailed product information, allowing customers to browse and search products easily.
- **Shopping Cart Management** – Enables users to add, remove, and update products in the shopping cart before placing an order.
- **Order Management** – Allows registered users to place orders securely, while the backend validates user authentication before processing orders.
- **Search Functionality** – Provides an efficient search feature that helps customers quickly find products based on their requirements.
- **Contact Management** – Enables customers to submit inquiries or feedback through the contact form, with messages stored securely in the database.
- **Responsive User Interface** – Offers a user-friendly and responsive interface that provides a seamless shopping experience across different devices.
- **Secure Data Management** – Stores product, user, order, and contact information in a centralized MySQL database using Spring Data JPA.
- **REST API Integration** – Facilitates smooth communication between the React frontend and Spring Boot backend through RESTful APIs.

Together, these capabilities make the Little Nest E-Commerce Website a secure, efficient, scalable, and user-friendly solution for modern online shopping.

6.PROPOSED METHOD

6.1 User Authentication

- Handles secure user registration and login for customers.
- Captures username, email, and password during registration.
- Encrypts passwords using Spring Security with BCrypt Password Encoder.
- Validates user credentials and prevents duplicate username registration.
- Allows only authenticated users to place orders.

6.2 Product Management

- Displays product listings with detailed product information.
- Allows customers to browse, search, and view products.
- Retrieves product data from the MySQL database through REST APIs.
- Provides a responsive interface for seamless product browsing.

6.3 Shopping Cart & Order Management

- Enables customers to add, update, and remove products from the shopping cart.
- Stores cart information using browser localStorage.
- Creates and stores orders for authenticated users.
- Validates user authentication before processing orders.
- Uses StoreOrder, OrderItem, and OrderController for order management.

6.4 Contact Management

- Provides a contact form for customers to send inquiries or feedback.
- Stores customer messages securely in the MySQL database.
- Processes contact requests through REST APIs.
- Maintains communication between customers and administrators.
- Uses ContactMessage and ContactController for handling customer messages.

6.5 System Administration

- Provides centralized management of products, users, orders, and customer messages.
- Maintains product and customer information in a single database.
- Ensures secure communication between the React frontend and Spring Boot backend.
- Supports efficient data retrieval using Spring Data JPA.

7. ALGORITHM

7.1 User Login

- User enters username and password on the login page.
- System validates that both fields are filled.
- Login request is sent to **AuthController → UserService**.
- System checks whether the username exists in the database.
- Entered password is verified using **BCrypt Password Encoder**.
- If the credentials are invalid, an error message is displayed.
- If authentication is successful, the user is logged in and can access the shopping features.

7.2 User Registration

- User enters username, email, and password on the registration page.
- System validates all required fields before submission.
- Registration request is sent to **AuthController → UserService**.
- System checks whether the username already exists.
- If a duplicate user is found, registration is rejected.
- If valid, the password is encrypted and the new user is saved using **UserRepository**.
- User is redirected to the login page after successful registration.

7.3 Product Search and Shopping Cart

- Customer browses or searches for a product.
- System retrieves matching products from the database.
- Customer selects a product and adds it to the shopping cart.
- Cart details are stored in browser localStorage.
- Customer can update product quantity or remove items before checkout.
- Cart is displayed with the latest selected products and total amount.

7.4 Order Processing

- Customer reviews the shopping cart and proceeds to checkout.
- Order request is sent to **OrderController → OrderService**.
- System verifies that the user is authenticated.
- If the user is not logged in, the order request is rejected.
- If authentication is successful, order details are stored in the database.
- Order confirmation is returned to the customer.

7.5 Contact Management

- Customer enters name, email, and message on the Contact page.
- System validates the input fields before submission.
- Request is sent to ContactController.
- Contact message is stored in the MySQL database using ContactMessageRepository.
- System confirms successful message submission to the customer.
- Administrators can later view and manage customer inquiries.

8.CODE IMPLEMENTATION

8.1 Login Module Code

Code: Contains authentication logic for validating user credentials and generating a JWT token after successful login.

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtil jwtUtil;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest request) {
    }
}
```

8.2 Registration Module Code

Code: Handles new user registration with encrypted password storage.

```
@PostMapping("/register")
public ResponseEntity<?> register(@RequestBody RegisterRequest request) {

    // Check duplicate username

    // Encrypt password

    // Save user in database

    // Return success response
}
```

8.3 Product Module code

Code: Retrieves product details from the database.

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductRepository repo;

    @GetMapping
    public List<Product> all() {
        return repo.findAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> get(@PathVariable Long id) {
        return repo.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
}
```

8.4 Order Module code

Code: Processes customer orders and stores order details.

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {
    @PostMapping
    @Transactional

public ResponseEntity<?> checkout(
            @Valid @RequestBody CheckoutRequest request) {
     // Validate authenticated user
```

```
    // Save order
    // Save ordered items
   // Return confirmation
    }
}
```

8.5Contact Module Code

Code: Stores customer inquiries submitted through the contact form.

```
@RestController
@RequestMapping("/api/contact-messages")
public class ContactController {

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Map<String, Object> create(
            @Valid @RequestBody ContactRequest request) {

        // Save contact message

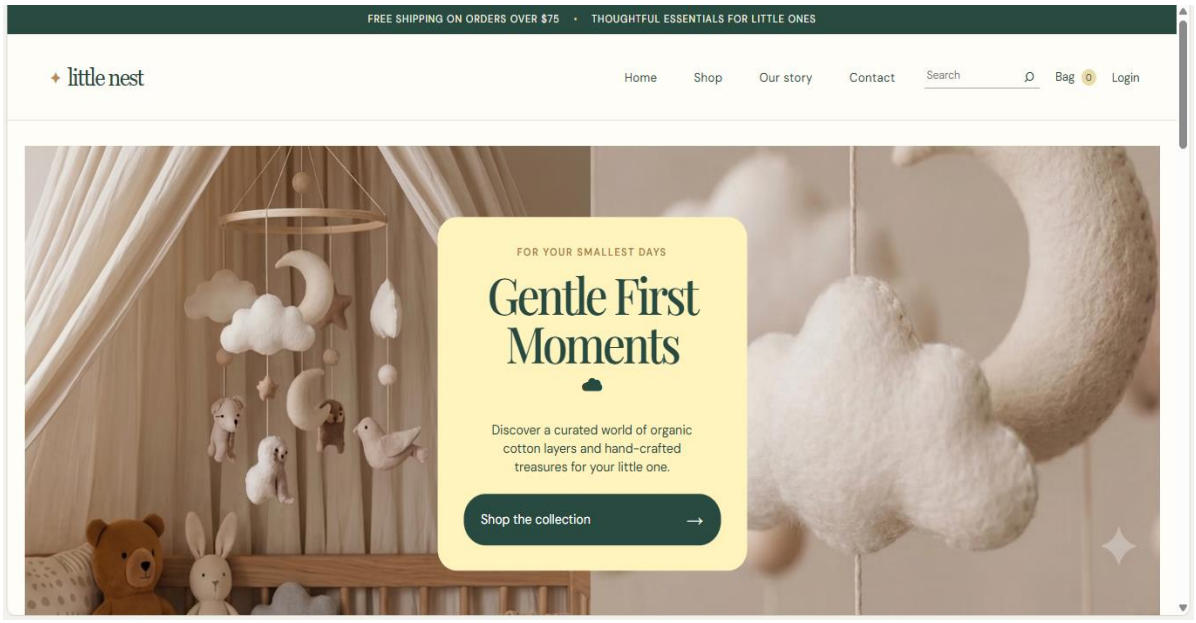
        // Notify administrator

        // Return success response
    }
}
```

9 SAMPLE OUTPUT

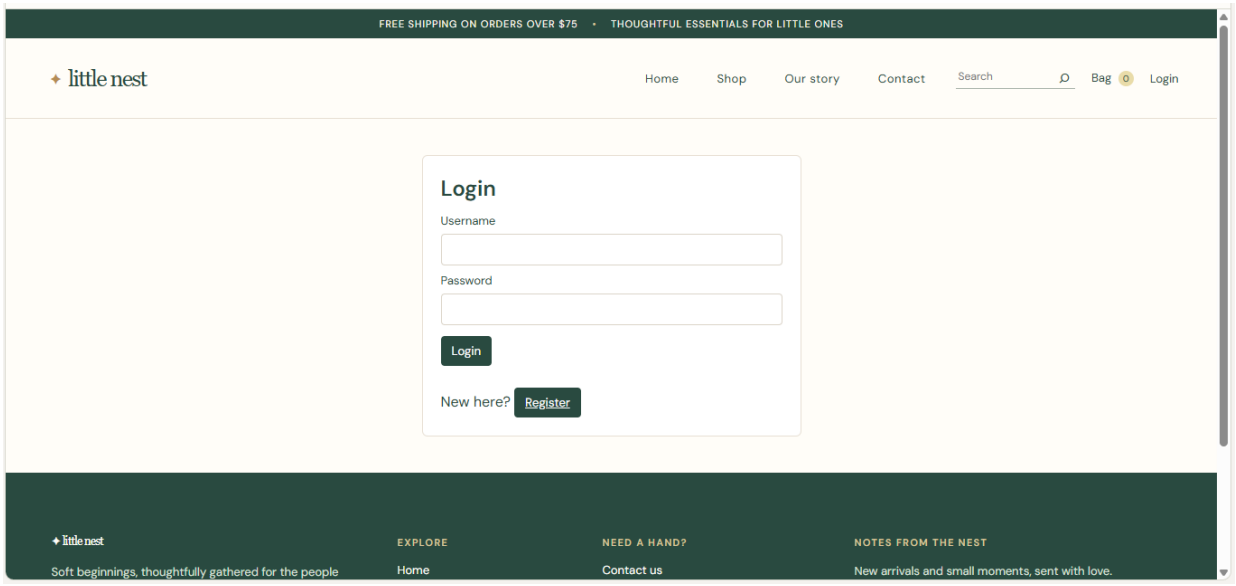
9.1 Home Page

Displays the main interface of the application.



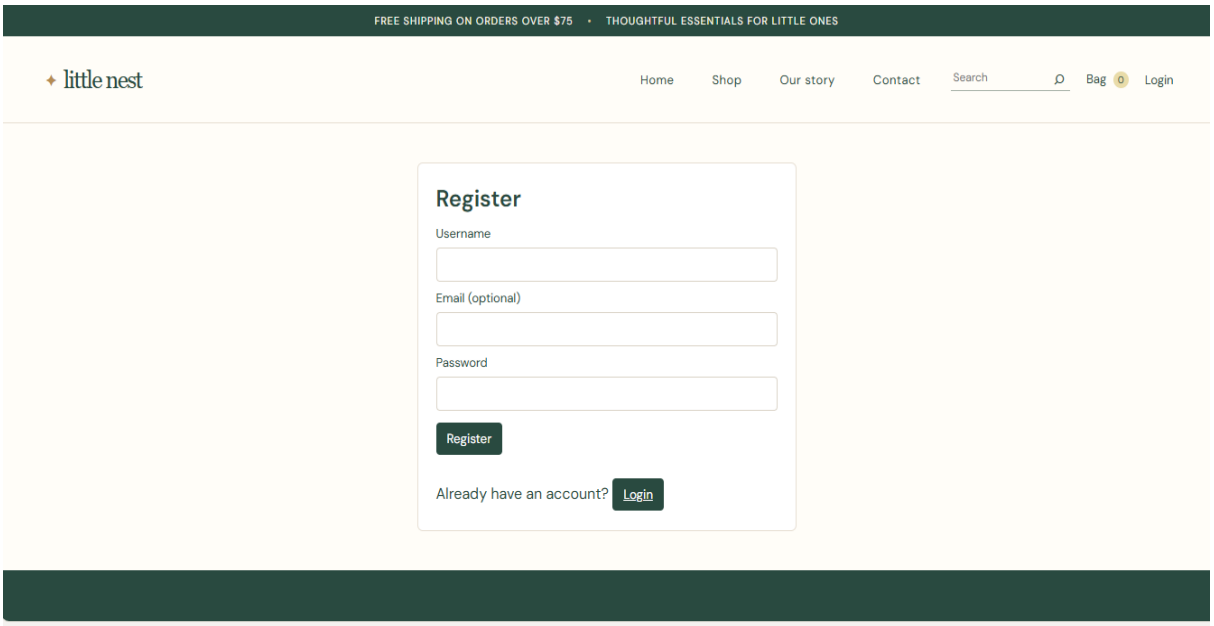
9.2 Login Page

Allows registered users to access their accounts.



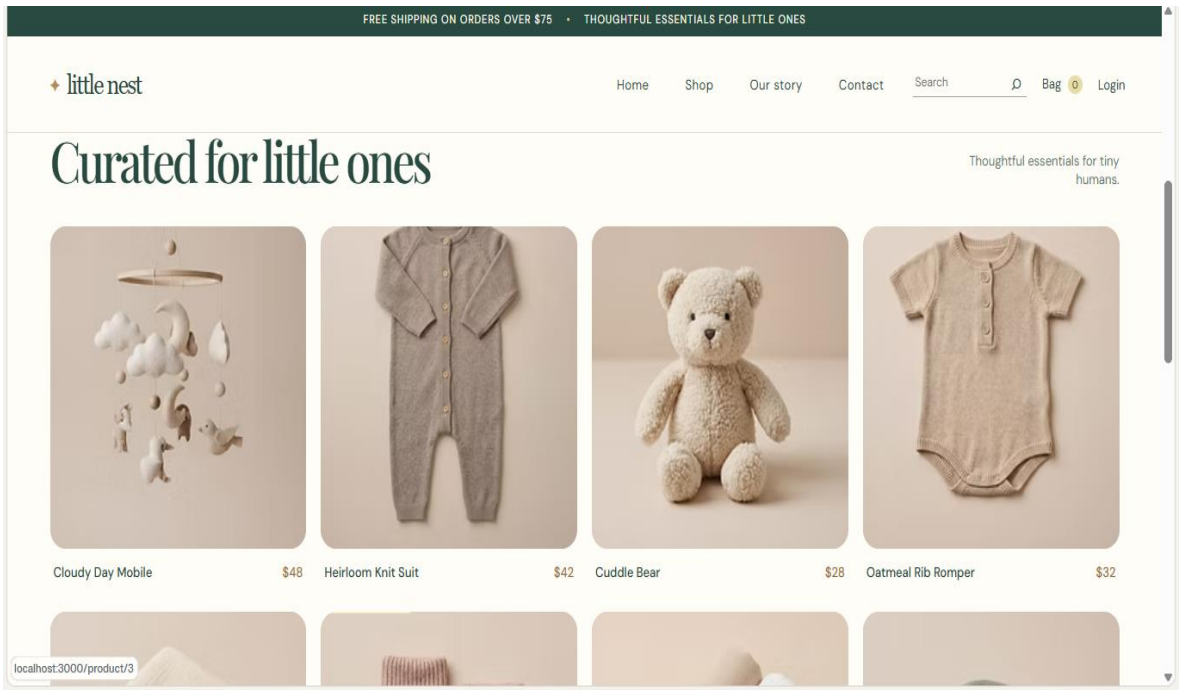
9.3 Registration Page

Allow new customers to create accounts.



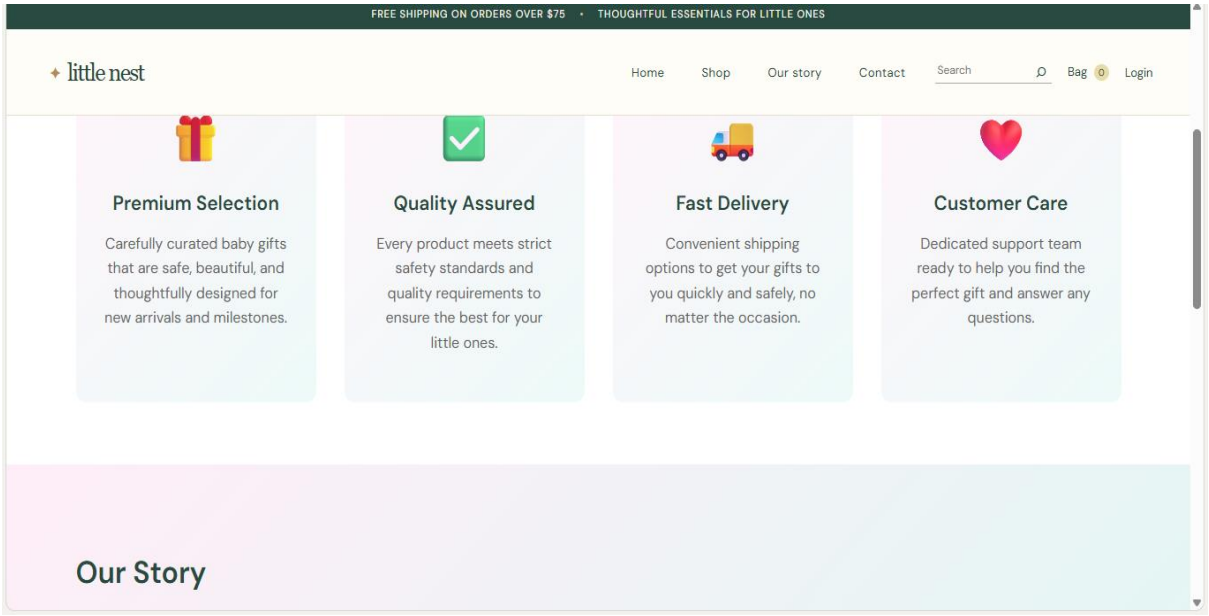
9.4 Product list page

Displays the product that all are available.



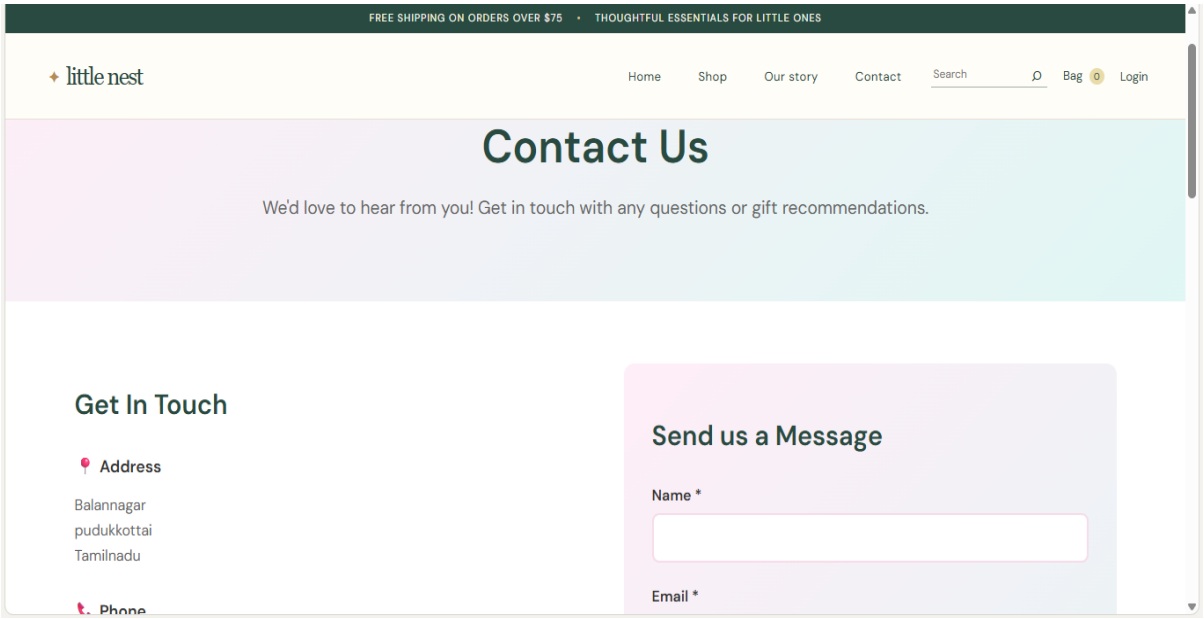
9.5 About page

The about page shows the story and details about the shop.



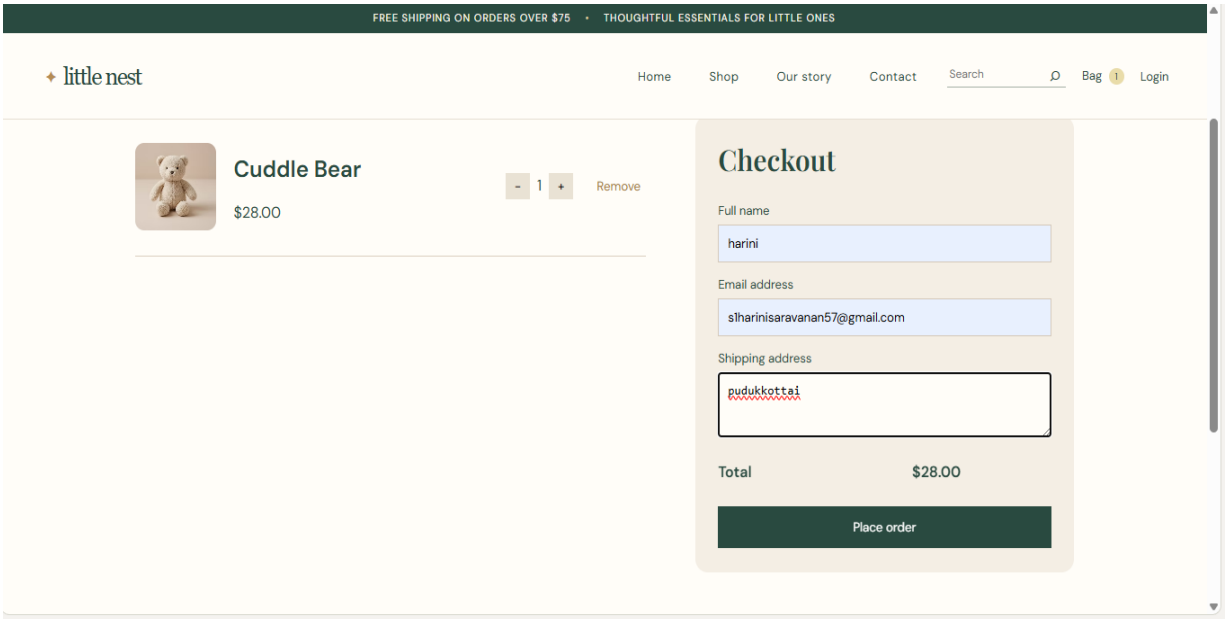
9.6 Contact page

Customer can send messages about their quires to the owner.



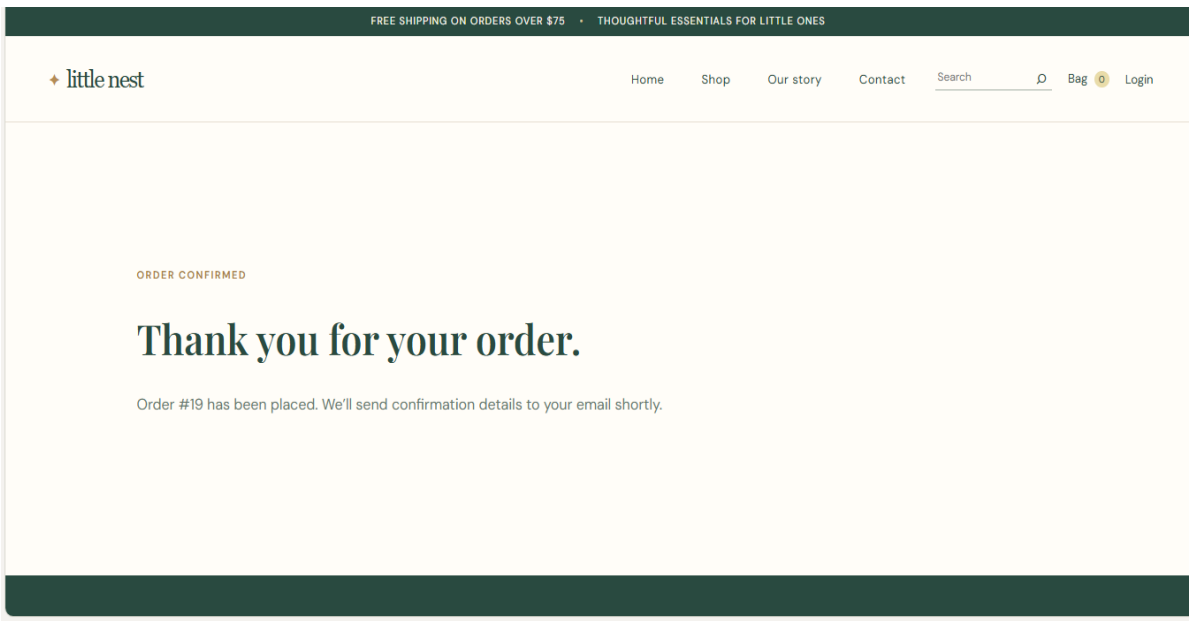
9.7Cart page:

Customer can place order in the cart by adding product to the cart.



9.8Order Confirmation page

Displays payment processing interface.



10. ANALYSIS

Testing of the **Little Nest E-Commerce Website** was carried out module by module to verify the correctness of the application's functionality and overall performance. Functional analysis confirmed that user registration, login, product browsing, product search, shopping cart management, order placement, and contact form submission worked correctly under both normal and invalid input conditions. Validation checks successfully handled scenarios such as duplicate usernames, invalid login credentials, empty input fields, and unauthorized order requests, ensuring the reliability of the system.

Performance analysis showed that the React frontend and Spring Boot backend communicated efficiently through RESTful APIs, providing quick response times for product retrieval, user authentication, and order processing. The MySQL database handled the storage and retrieval of product, user, order, and contact information effectively, while Spring Data JPA simplified database operations and improved application performance.

Usability analysis indicated that the responsive React-based interface provided a smooth and intuitive shopping experience across different screen sizes. The contact module also allowed users to submit inquiries easily, improving communication between customers and the store.

A comparison with traditional shopping methods showed significant improvements in terms of accessibility, efficiency, and security. Overall, the analysis confirms that the **Little Nest E-Commerce Website** successfully achieves its objectives by providing a secure, responsive, and user-friendly online shopping platform while reducing manual effort and improving the overall customer experience.

11. CONCLUSION

The **Little Nest E-Commerce Website** successfully demonstrates the implementation of a modern Java Full Stack web application for online shopping. By replacing traditional shopping methods with a centralized digital platform, the system enables customers to browse products, search for items, manage their shopping cart, and place orders conveniently. The application improves operational efficiency, enhances customer satisfaction, and provides a secure shopping experience through user authentication and efficient data management.

The project adopts a client-server architecture with a responsive **React.js** frontend and a robust **Spring Boot** backend that communicate through RESTful APIs. Core modules such as user authentication, product management, shopping cart, order management, and contact management were successfully designed, implemented, and tested. The use of **Spring Security**, **Spring Data JPA**, and **MySQL** ensures secure authentication, reliable data storage, and efficient database operations.

This internship project provided valuable hands-on experience in Java Full Stack Development, REST API development, database management, frontend-backend integration, and software testing. It strengthened practical knowledge of modern web technologies and software development practices while providing a solid foundation for developing scalable, secure, and real-world e-commerce applications.

12. FUTURE WORK

- Integration of a secure online payment gateway (e.g., **Razorpay** or **Stripe**) for real-time payment processing.
- Implementation of **JWT-based authentication** with role-based authorization for enhanced application security.
- Addition of an **order tracking** feature that allows customers to monitor the status of their orders in real time.

- Integration of **email and SMS notifications** for user registration, order confirmation, shipping updates, and password recovery.
- Implementation of an **inventory management module** to automatically update product stock and notify administrators of low inventory levels.
- Development of **product review and rating** functionality to improve customer engagement and support better purchasing decisions.
- Deployment of the application on cloud platforms such as **AWS**, **Microsoft Azure**, or **Google Cloud Platform (GCP)** to improve scalability, availability, and reliability.
- Enhancement of the application using **AI-based product recommendation systems** to provide personalized shopping suggestions based on customer preferences and purchase history.

13. REFERENCES

1. React Documentation – Official guide for building user interfaces using React.js. https://react.dev/
2. Spring Boot Official Documentation – Reference guide for Spring Boot application development and REST API implementation. https://spring.io/projects/spring-boot
3. Spring Security Reference Documentation – Official documentation for implementing authentication, authorization, and password encryption. https://spring.io/projects/spring-security
4. MySQL Documentation – Official documentation for MySQL database management and SQL reference. https://dev.mysql.com/doc/
5. Oracle Java Documentation – Official Java language documentation and standard library reference. https://docs.oracle.com/en/java/
6. Spring Data JPA Reference Documentation – Official guide for database access and repository implementation using Spring Data JPA. https://spring.io/projects/spring-data-jpa

7. React Router Documentation – Official documentation for client-side routing in React applications. https://reactrouter.com/
8. Axios Documentation – Official documentation for making HTTP requests from the React frontend. https://axios-http.com/
9. W3Schools – Learning resource for HTML, CSS, JavaScript, and web development concepts. https://www.w3schools.com/
10. GeeksforGeeks – Tutorials and examples for Java, Spring Boot, React, and full-stack web development. https://www.geeksforgeeks.org/